Experiment No. 1

Aim/Objective :

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem. Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory :

Water-Jug and Missionaries & Cannibals both problem are classic example of state-space problems used in Artificial Intelligence. In such problem each possible combination is represented using state. And traversing between state is done using defined operation.

In water-jug problem each state represent amount of water in two jugs and operation allowed are filling, emptying and pouring. The goal is to have required amount of water in jug while not exceeding the capacity of jugs.

In missionaries & cannibals the state is represent number of missionaries and cannibals on left side and position of boat. The goal is to transport all 3 missionaries & cannibals initially on left side to right side of river while ensuring that at any point missionaries are not outnumbered by cannibals on either side of river.

Implementation :

Water-Jug Problem

```
def fillx(a,state) :          #To fill jug A
  print("You have called fill x")
  state[0] = a
  return state

def filly(b,state) :          #To fill jug B
  print("You have to called fill y")
```

```
  state[1] = b
  return state

def emptyx(state) :           #To empty jug A
  print("`You have called empty x")
  state[0] = 0
  return state

def emptyy(state) :           #To empty jug B
  print("You have called empty y")
  state[1] = 0
  return state

def pourxy(a,b,state) :       #To pour water from jug A to B
  print("You have called pour x to y")
  remy = b - state[1]
  if state[0] > remy :
    state[0] = state[0] - remy
    state[1] = b
  else :
    state[1] = state[1] + state[0]
    state[0] = 0
  return state

def pouryx(a,b,state) :        #To pour water from jug B to A
  print("You have called pour y to x")
  remx = a - state[0]
  if(state[1] > remx) :
    state[1] = state[1] - remx
    state[0] = a
  else :
    state[0] = state[0] + state[1]
    state[1] = 0
  return state


a,b = map(int,input("Enter capacities of JUGS").split())
g = int(input("Enter goal to reach"))

state = [0,0]
goals = []
```

```
if(g<=a) :
  goals.append([g,0])
if(g<=b) :
  goals.append([0,g])

print("Initial State :",state,"\n")
print("Options :-")
print("1 : Fill x")
print("2 : Fill y")
print("3 : Empty x")
print("4 : Empty y")
print("5 : Pour x to y")
print("6 : Pour y to x")

# Taking input from user for next operation until goal state is reached
while(state not in goals) :
  choice = int(input("Enter your choise"))
  if(choice == 1) :
    state = fillx(a,state)
  elif(choice == 2) :
    state = filly(b,state)
  elif(choice == 3) :
    state = emptyx(state)
  elif(choice == 4) :
    state = emptyy(state)
  elif(choice == 5) :
    state = pourxy(a,b,state)
  elif(choice == 6) :
    state = pouryx(a,b,state)
  else :
    print("Invalid Choise")

  print("Current State :",state,"\n")

print("Goal Reached")
```

Missionaries & Cannibals

```
from collections import deque

def is_valid(m,c) :       #To check if current state is valid

  return (m == 0 or m>=c) and (3-m == 0 or 3-m>=3-c)


def missioanries_cannibals() :

  state = (3,3,0)       # Defining initial and goal state

  goal = (0,0,1)

  operations = [(1,0),(2,0),(0,1),(0,2),(1,1)] #possible o/p on boat

  queue = deque()

  queue.append((state,[state]))

  visited = set()

# Run until goal state is reached or there is no more state to explore

while queue :

    (m,c,b),path = queue.popleft()


    if (m,c,b) == goal :

      print("Success")

      for state in path :

        print(state)

      return

 #To prevent repetitive iteration between same states

    if (m,c,b) in visited :

      continue

    visited.add((m,c,b))


    for mb,cb in operations :

      if(b == 0) :
```

```
        nm = m-mb
        nc = c-cb
      else :
        nm = m+mb
        nc = c+cb
#Checking implicit and explicit constraint to prevent invalid states
      if 0<=nm<=3 and 0<=nc<=3 :
        if is_valid(nm,nc) :
          queue.append(((nm,nc,1-b),path + [(nm,nc,1-b)]))


missioanries_cannibals()
```

Missionaries & Cannibal (With Boat Size 3)

```
from collections import deque
def is_valid(m,c) :       #To check if current state is valid
  return (m == 0 or m>=c) and (3-m == 0 or 3-m>=3-c)


def missioanries_cannibals_capacity3() :
  state = (3,3,0)       # Defining initial and goal state
  goal = (0,0,1)
 operations = [(3,0),(2,1),(1,2),(0,3),(2,0),(0,2),(1,1),(1,0),(0,1)]


  queue = deque()
  queue.append((state,[state]))
  visited = set()


# Run until goal state is reached or there is no more state to explore
while queue :
```

```
        (m,c,b),path = queue.popleft()


        if (m,c,b) == goal :
          print("Success")
          for state in path :
            print(state)
          return
      #To prevent repetitive iteration between same states
        if (m,c,b) in visited :
          continue
        visited.add((m,c,b))


        for mb,cb in operations :
          if(b == 0) :
            nm = m-mb
            nc = c-cb
          else :
            nm = m+mb
            nc = c+cb
     #Checking implicit and explicit constraint to prevent invalid states
          if 0<=nm<=3 and 0<=nc<=3 :
            if is_valid(nm,nc) :
              queue.append(((nm,nc,1-b),path + [(nm,nc,1-b)]))


     missioanries_cannibals_capacity3()
```

Water-Jug Problem Using BFS

```
from collections import deque
```

```
def water_jug(a,b,g) :
  state = (0,0)  #Defining initial and goal states
  goal = set()
  if g <= a :
    goal.add((g,0))
  if g <= b :
    goal.add((0,g))

  queue = deque()
  queue.append((state,[state]))
  visited = set()
# Run while loop until goal state is achieved
  while queue :
    (x,y),path = queue.popleft()

    if (x,y) in goal :
      print("Success")
      for state in path :
        print(state)
      return

    if (x,y) in visited :
      continue
    visited.add((x,y))

    #Next possible states after performing all 6 operation on current state
```

```
    states = [(a,y),(x,b),(0,y),(x,0),(x - min(x,b-y),y + min(x,b-y)),(x +
min(y,a-x),y - min(y,a-x))]

    for state in states :

      queue.append((state,path+[state]))


  print("No solution exist")


water_jug(5,3,4)
```

Output/Result :

1) Water - Jug

```
...  Enter capacities of JUGS3 5
     Enter goal to reach4
     Initial State : [0, 0]

     Options :-
     1 : Fill x
     2 : Fill y
     3 : Empty x
     4 : Empty y
     5 : Pour x to y
     6 : Pour y to x
     Enter your choise1
     You have called fill x
     Current State : [3, 0]

     Enter your choise5
     You have called pour x to y
     Current State : [0, 3]

     Enter your choise1
     You have called fill x
     Current State : [3, 3]

     Enter your choise5
     You have called pour x to y
     Current State : [1, 5]

     Enter your choise4
     You have called empty y
     Current State : [1, 0]

     Enter your choise5
     You have called pour x to y
     Current State : [0, 1]

     Enter your choise1
     You have called fill x
     Current State : [3, 1]

     Enter your choise5
     You have called pour x to y
     Current State : [0, 4]
```

2) Missionaries & Cannibals

```
Success
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
```

3) Missionaries & Cannibals (With boat capacity 3)

```
Success
(3, 3, 0)
(3, 0, 1)
(3, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 0, 1)
```

4) Water - Jug using BFS

```
Success
(0, 0)
(5, 0)
(2, 3)
(2, 0)
(0, 2)
(5, 2)
(4, 3)
(4, 0)
```

Conclusion :

Through this experiment, I understood about state-space solving problem using classical toy AI problem such as Water-Jug Problem and Missionaries & Cannibals Problem. And how real world problem can be modeled and solved using computational tool.

I learned how problem can be represented using simple state and how different actions such as filling, emptying and pouring changes the state in water-jug problem.

In missionaries & cannibals problem I learned how we can use BFS explore the state-space step by step and how the constraints are used to prevent system from entering any invalid states and traverse through only safe configuration to reach the goal efficiently

Thought Provoking Question :

1) Why is BFS preferred over DFS here ?

Because BFS explore all possible state at each level, where as DFS explore particular path until solution is found or we reach invalid state, as a result BFS will always provide solution with least number of intermediate steps.

Since BFS check solution step by step it is sure to provide solution if it exist, on other hand DFS is more likely to get struck into invalid or repetitive states.

2) What happens if boat capacity is increased to 3?

In such case the number of combinations of missionaries and cannibals travelling in boat increases thus increasing the branching factor.

But increasing the boat size allows to move all missionaries and cannibals at once thus reducing the complexity of the problem, resulting in fewer intermediate steps.

3) Can the water jug problem be solved using BFS instead of manual input.

Yes water jug problem can be solved using BFS. Similar to Missionaries & Cannibals it can be done by defining initial and goal state and iterating until goal state is reached. But unlike Missionaries & Cannibal where operations are fixed, in water – jug problem next state depends on the current state.

4) How do constraints prune invalid states.

For each intermediate state when next state are calculated based on operation, the next state is first verified if it passes both implicit and explicit constraints set by problem before adding them to queue, thus preventing system to enter any invalid state.